MINI PROJECT / REPRODUCE + EXTEND

Predicting Bazel Build Times

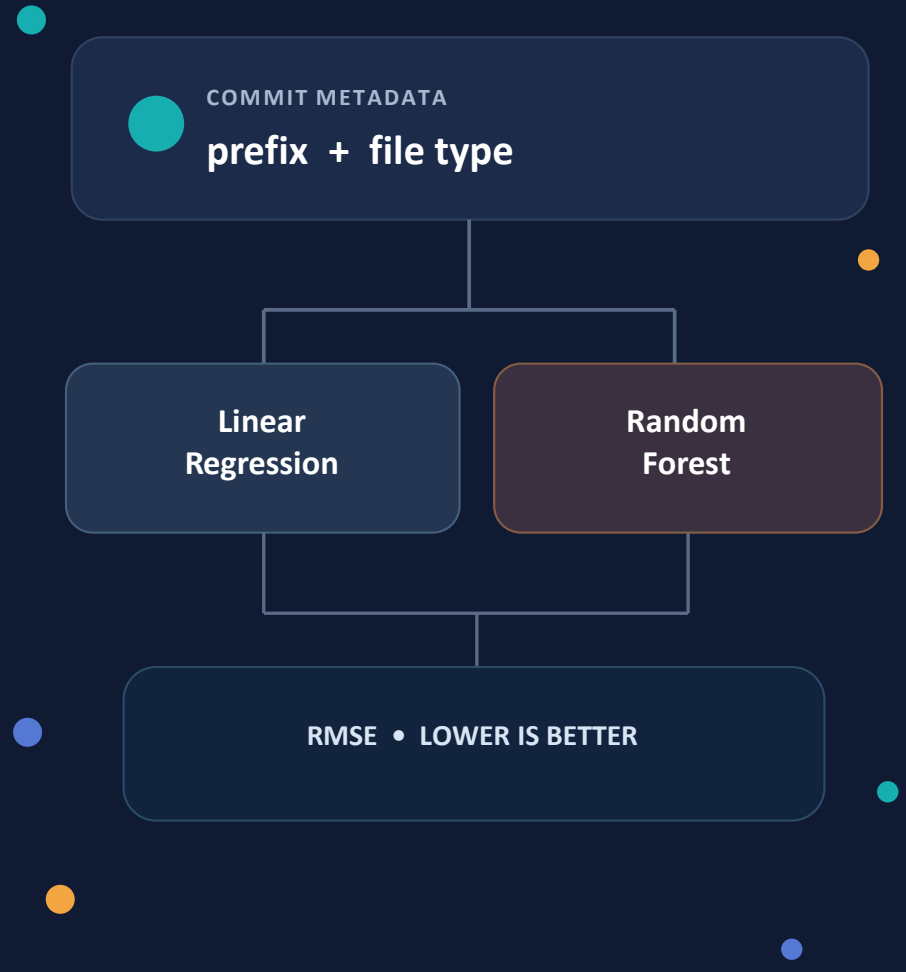
Reproducing the reference pipeline and testing a Random Forest extension

REFERENCE MODEL • LINEAR REGRESSION

ADD-ON • RANDOM FOREST

Reference paper: Muhammad Umar Nadeem & Sarah Raza

Dataset: 500 Bazel commit/build examples



REPRODUCE → EXTEND → COMPARE

Can a commit hint at the cost of a build?

Goal: estimate the CPU time required for a Bazel build from lightweight information about changed files.

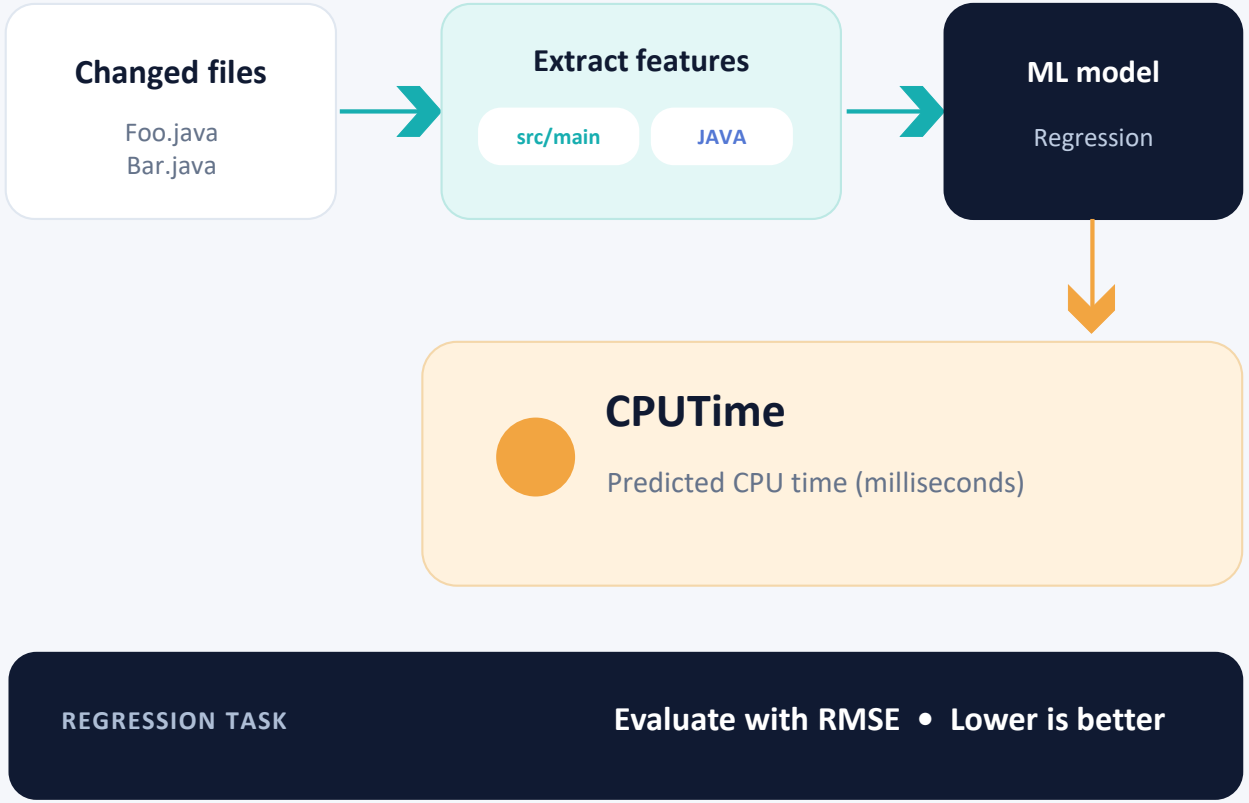
THE PROBLEM

Build time is a resource cost.

The reference explores whether a commit's changed-file metadata can predict CPU time, helping developers reason about build cost and efficiency.

- Input: most common file-path prefix
- Input: most common file type
- Target: CPUTime in milliseconds

FROM COMMIT TO PREDICTION



REGRESSION TASK

Evaluate with RMSE • Lower is better

500 commit/build examples, two categorical inputs

The supplied processed data comes from the open-source bazelbuild/bazel project.

500
COMMIT EXAMPLES

300 / 100 / 100
TRAIN / VALIDATION / TEST

8
OBSERVED PREFIX CATEGORIES

4
OBSERVED FILE TYPES

What each row contains

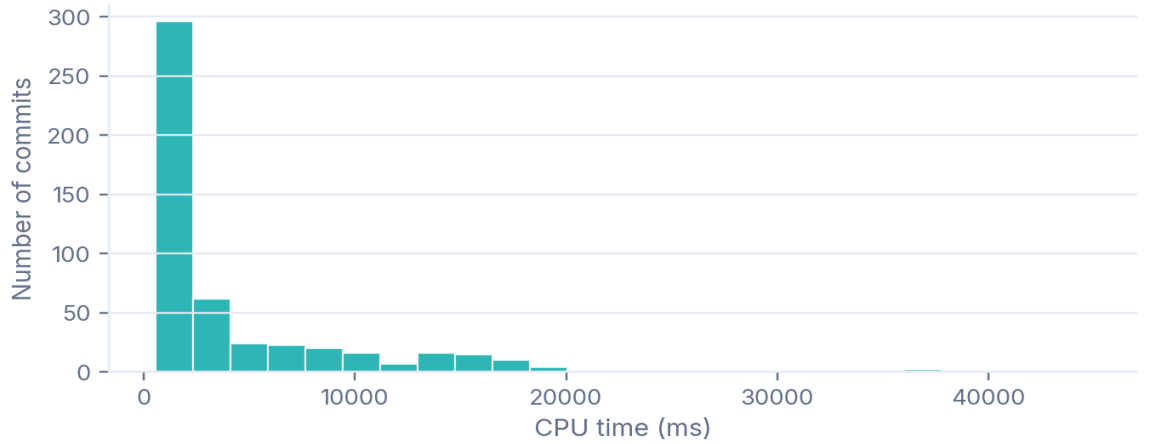
PREFIX — **src/main**
Most common path prefix in the commit

TYPE — **JAVA**
Most common file type in the commit

TARGET — **CPUTime (ms)**

Each sample maps those two features to a build CPU-time measurement.

CPU build time is right-skewed

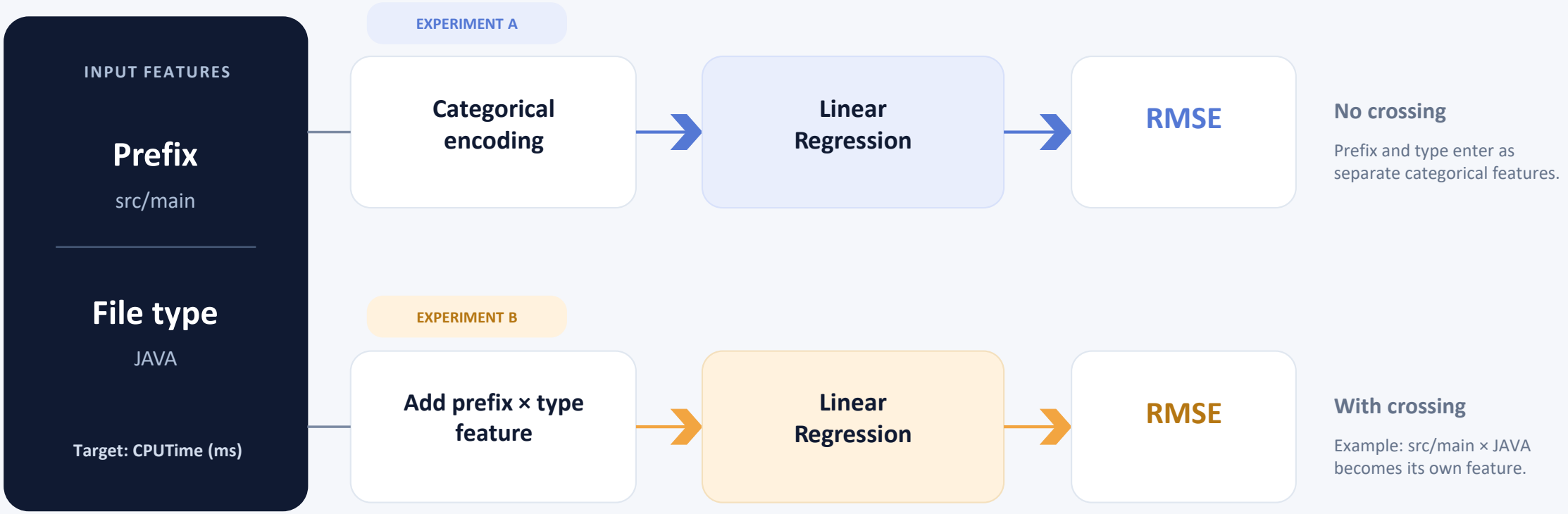


Mean: 4,299.40 ms **Median: 1,837.5 ms** **Range: 564–44,863 ms**

Data linkage assumption: the two CSVs have no shared commit key, so input and target rows were paired in order (target IDs 1–500).

The reference compares two linear-regression pipelines

Feature crossing gives the model an explicit signal for a prefix/type combination.



Reference settings	SGD	Learning rate 0.01	Batch size 16	9 training periods	RMSE

Reproduce the experiment, not just the reported number

We independently rebuilt the pipeline from the paper and the supplied CSVs.

1

Load & align

Read 500 rows from each CSV; pair rows in the supplied order.

2

Encode categories

One-hot encode prefix and type; add prefix × type block for experiment B.

3

Train the model

Linear regression trained with mini-batch SGD over 9 periods.

4

Measure error

Calculate training, validation and test RMSE at each period.

EXPERIMENT SETTINGS

SPLIT	LEARNING RATE	BATCH SIZE	PERIODS	METRIC
300 / 100 / 100	0.01	16	9	RMSE

Reproducibility note The paper omits the exact split seed and grid-search candidates; we used row order and the reported selected hyperparameters.

Our reproduced training RMSE is close to the paper

Values below are average training RMSE over the 9 periods shown in the reference paper.

EXPERIMENT	PAPER	REPRODUCED	DIFF.
Linear Regression without crossing	5,735.84	5,707.02	−0.50%
Linear Regression with prefix × type crossing	5,628.09	5,685.62	+1.02%

RMSE is in milliseconds

REPRODUCTION CHECK

~0.5%

difference without crossing

~1.0%

difference with feature crossing

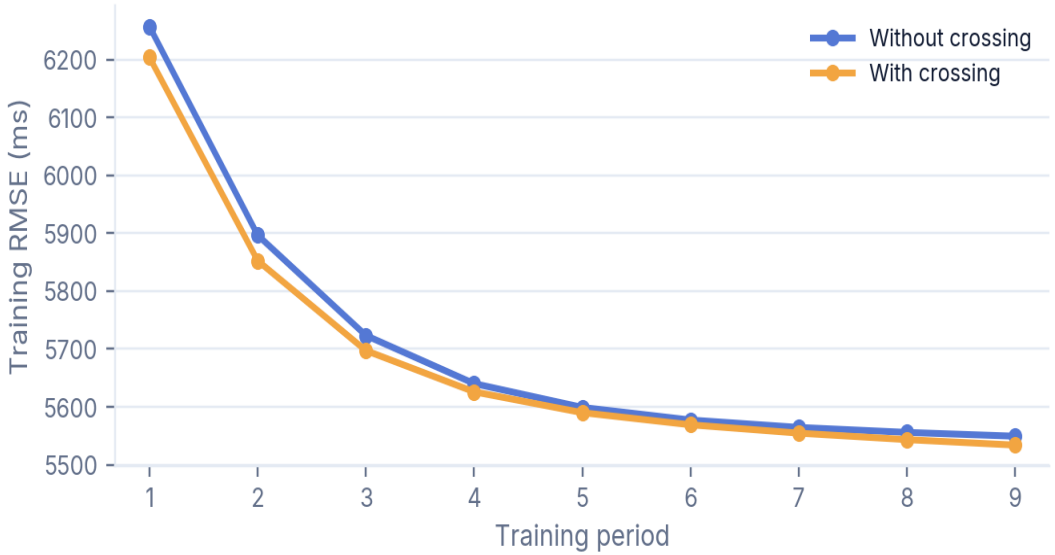
Takeaway **Both reproduced averages are within about 1% of the paper's reported training RMSE.**

Important: these are training-set averages, not the held-out test values used for the Part 2 comparison.

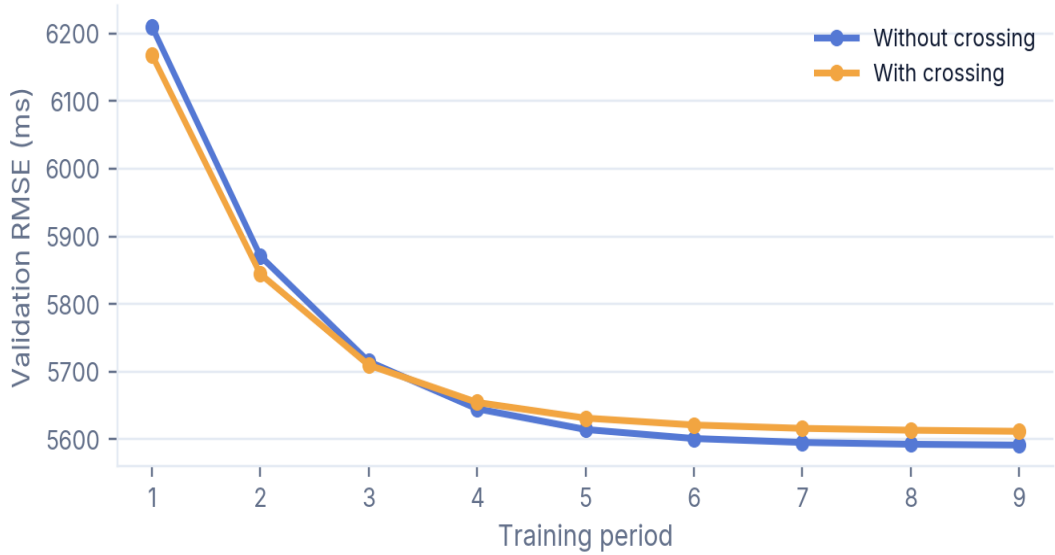
Error drops quickly, then starts to plateau

The curves show how the linear models behave over the 9 training periods.

Training error falls, then flattens



Validation curves converge



WHAT TO NOTICE

Crossing lowers training RMSE, but validation curves are close and favour the non-crossed model slightly in later periods.

Add-on model: Random Forest Regressor

A different model family gives us a meaningful extension beyond the linear reference model.

WHY RANDOM FOREST?

A nonlinear, tree-based alternative

- Can represent nonlinear relationships and feature interactions.
- Works with a compact tabular representation of categorical inputs.
- Chosen for methodological contrast — not because it was expected to win.

EXPERIMENT SETUP

TREES	RANDOM STATE
300	42
MAX FEATURES	**MIN LEAF SIZE**
1.0	1

Same 300 / 100 / 100 split

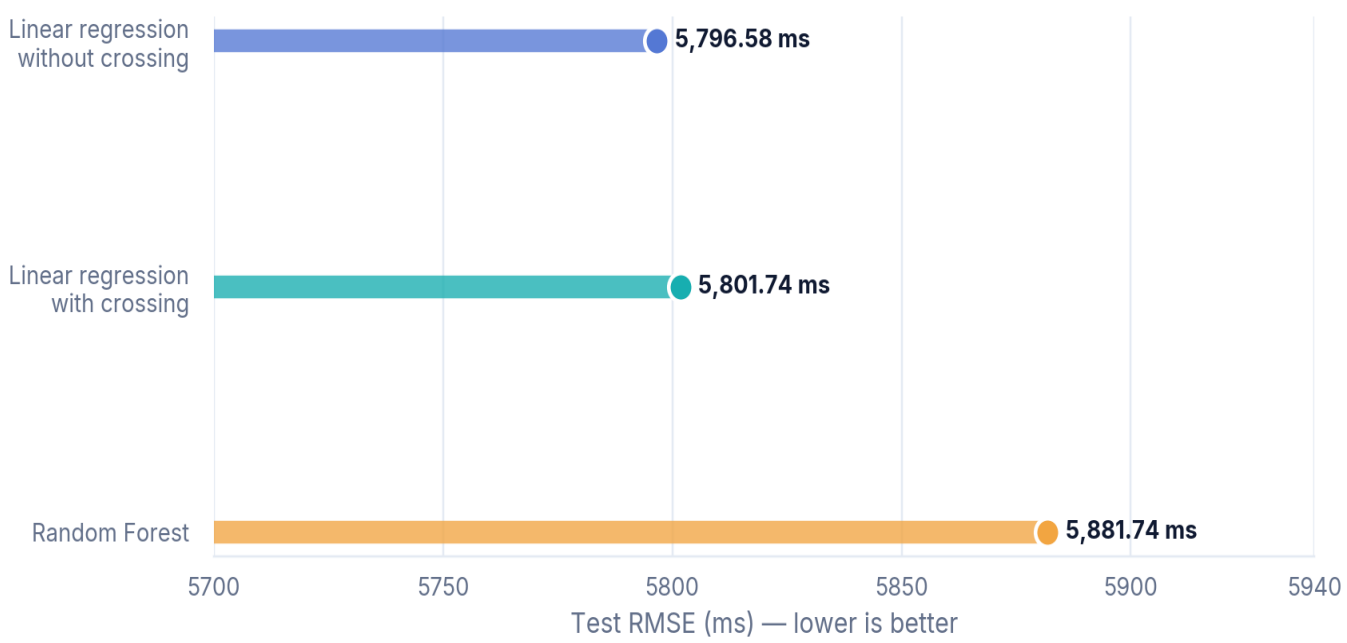
Same prefix + type + prefix × type representation

Same target (CPUTime) and same metric (RMSE)

TRAIN RMSE	VALIDATION RMSE	TEST RMSE
5,382.47 ms	5,858.90 ms	5,881.74 ms

On the shared test set, Random Forest was worse

All three scores use the same 100-example test split; lower RMSE indicates better prediction.



Zoomed axis: 5,700–5,940 ms

RANDOM FOREST RESULT

5,881.74

ms test RMSE

+85.16 ms / +1.47%

vs. Linear Regression without crossing

+79.99 ms / +1.38%

vs. Linear Regression with crossing

Fair-comparison note: the paper's headline numbers are 9-period training averages; this chart uses final-period test RMSE.

What the results do — and do not — tell us

The extra model is a valid extension even though it did not outperform the reference models.

KEY FINDINGS

- The reproduction closely matches the paper's training-RMSE averages (differences: about 0.50% and 1.02%).
- Feature crossing lowered average training RMSE by about 21 ms in our run, but its final test RMSE was 5.16 ms higher than the non-crossed model.
- Random Forest had the highest test RMSE; extra model flexibility did not improve generalization on this split.

LIMITATIONS

- Only 500 examples are available, and the inputs summarize commits with just two categorical features.
- The supplied CSVs do not share a commit key; row-order alignment is an assumption.
- The paper omits the exact data-split rule/seed and the candidates used in its grid search.

Interpret results for this dataset, feature representation and split — not as a universal ranking of models.

Reproduce carefully. Extend fairly.

The value is in a transparent comparison — not in forcing the add-on model to win.

01	**Reproduction**	Both reproduced training-RMSE averages are within about 1% of the paper.
02	**Feature crossing**	It helped training fit; final test RMSE was nearly the same, slightly higher.
03	**Extension**	Random Forest was worse on this shared test set by about 1.4%.

REFERENCE PAPER

Muhammad Umar Nadeem & Sarah Raza

“Predicting Bazel Build Times Using Machine Learning”

DATA / ORIGINAL MATERIALS

github.com/umar-nadeem/CS229-Final-Project

Evaluation metric: RMSE (milliseconds)

Thank you • Questions?